CUDA Graphs in Megatron-Core

Capture once, replay every step, at three scopes

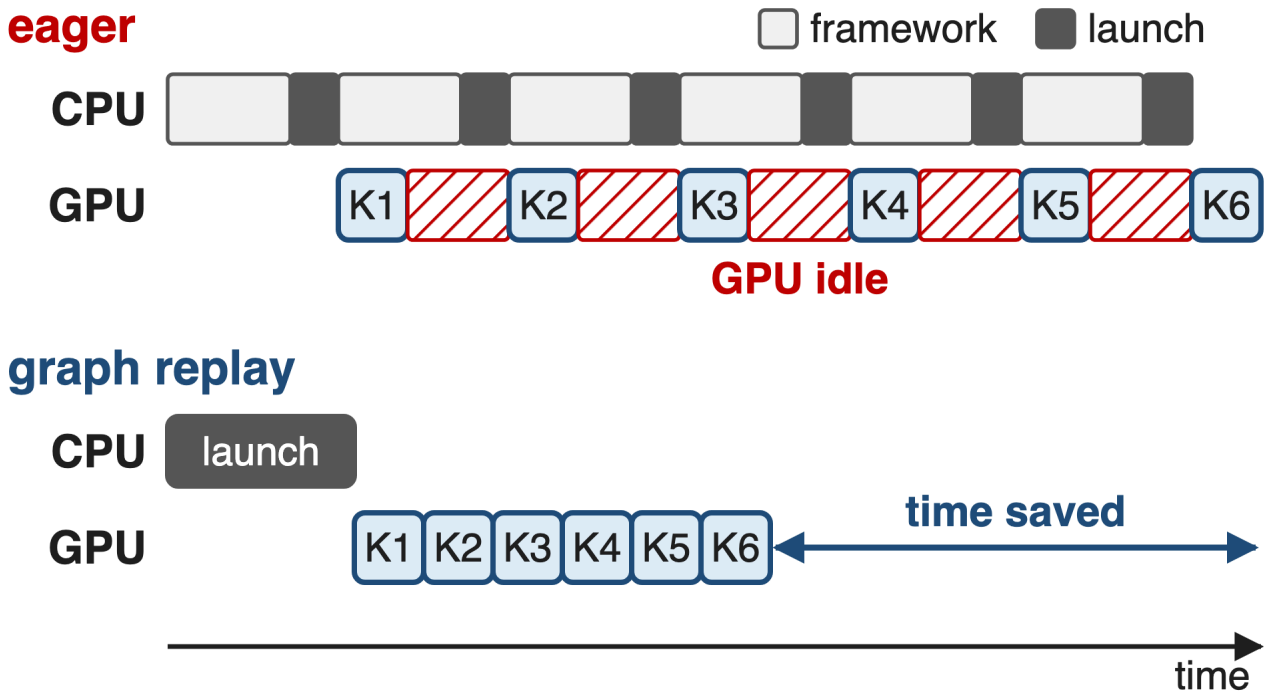
NVIDIA · Megatron-Core (--cuda-graph-impl) · Transformer Engine

Explainer deck · Industrial Pretraining Architecture & Infra Tech Landscape (2026)

Kernel Fusion & Runtime · github.com/NVIDIA/Megatron-LM

Small kernels outrun the CPU that launches them

- Per op: Python, framework, launch (~1–5 µs per kernel)
- MoE: routers, permutes, ~128-token expert GEMMs
- FP8/FP4: extra quantize kernels, faster GEMMs



Eager FP4 training on GB300 runs 1.79× slower

1.79×

speedup from graphing the whole step: Llama 3.1 405B, FP4, 512 GB300
(1.15× on B200, 1.54× on GB200; FP8 shows none)

NVIDIA dl-cuda-graph docs, "Llama 3.1 405B" example (MLPerf Training v5.1)

Training breaks every rule a graph needs

a captured graph needs		a training iteration has
fixed tensor shapes	✗	dropless MoE: tokens per expert vary
no host sync inside	✗	D2H token counts and NaN checks
fixed input addresses	✗	fresh dataloader tensors every step
replay in capture order	✗	PP interleaves microbatches in one pool
same work on every replay	✗	FP8 amax reduce once per step, not per layer

Naive capture either fails or silently replays stale data

Fusion and plain graphs only go halfway

- **Kernel fusion** (deck 42)
- Fewer launches per layer
- **Per-op host cost remains**

- **Plain PyTorch graphs**
- Assume static everything
- **No FP8 state, no PP order**

Needed: graph everything static, replayed in the order training runs

Insight: almost everything in a step is static

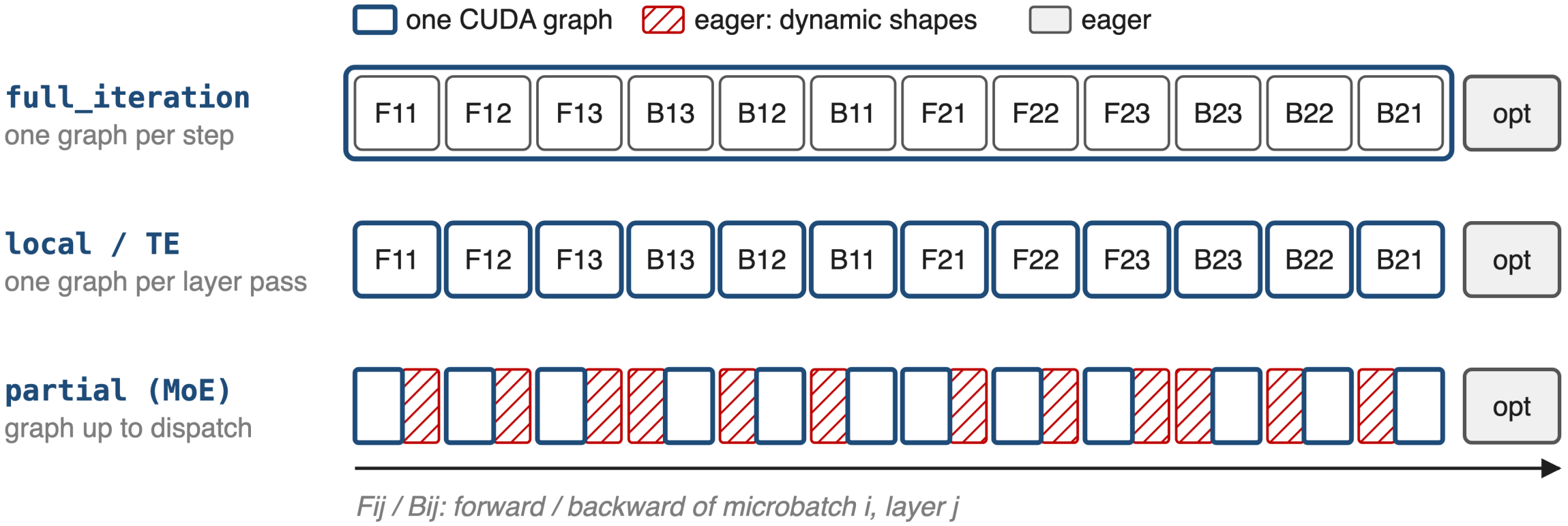
- Dense or drop-and-pad MoE: the whole forward-backward
- Dropless MoE: only dispatch → experts → combine vary

Insight: almost everything in a step is static

- Dense or drop-and-pad MoE: the whole forward-backward
- Dropless MoE: only dispatch → experts → combine vary

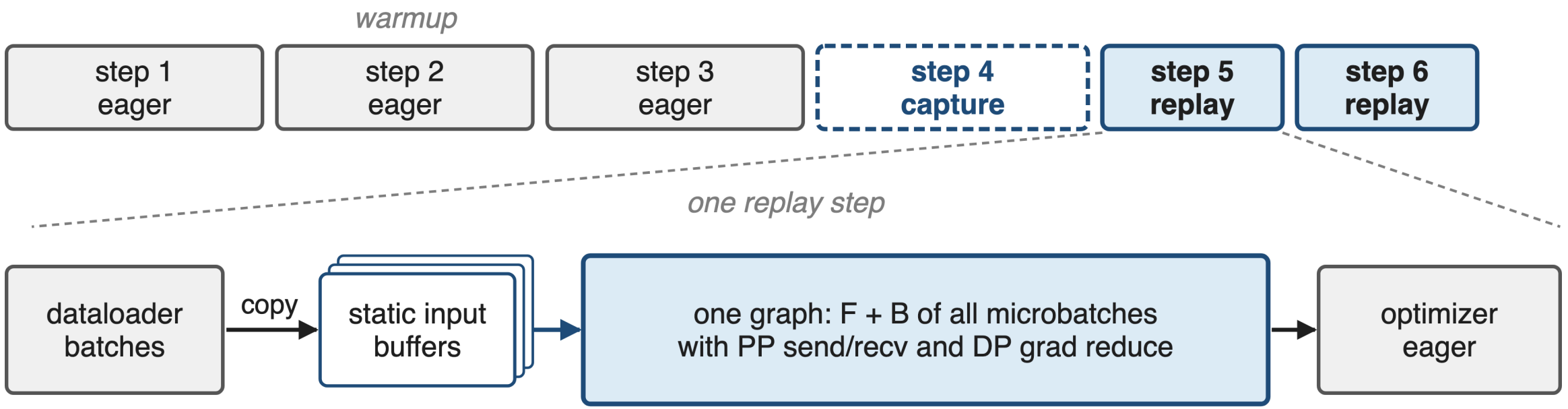
Graph the largest static scope; replay graphs in capture order

Three implementations, three capture scopes



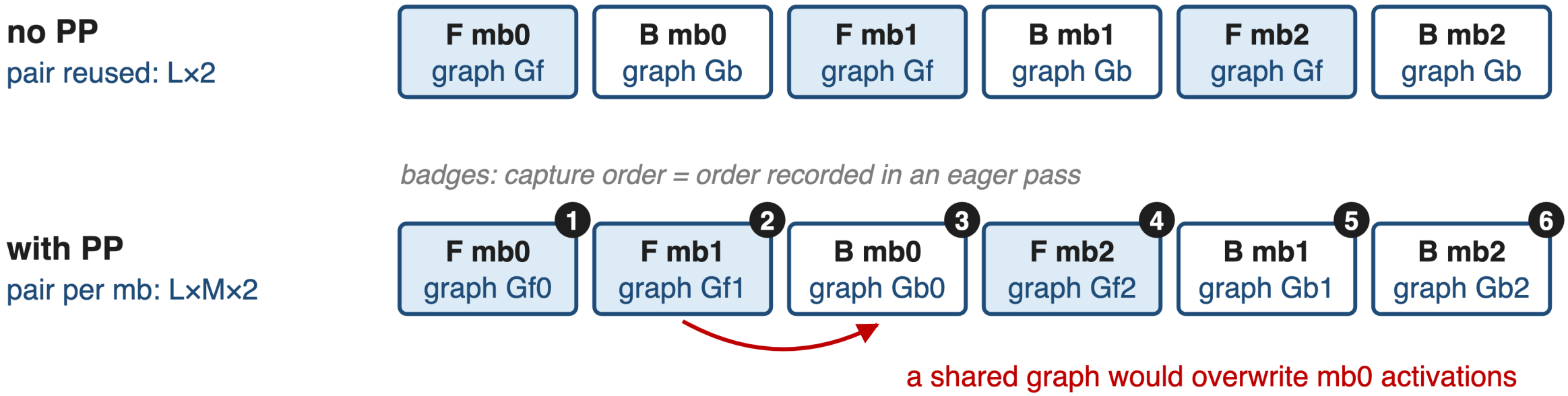
--cuda-graph-impl full_iteration | local | transformer_engine; --cuda-graph-modules narrows per-layer capture

❶ full_iteration: one graph per training step



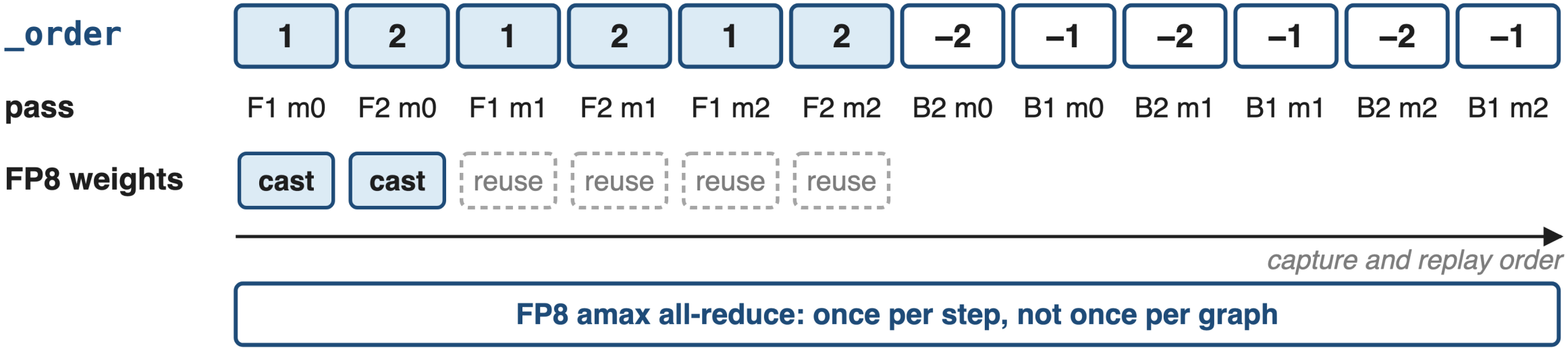
Requires static shapes everywhere and --no-check-for-nan-in-loss-and-grad

❷ local: record the order, then capture



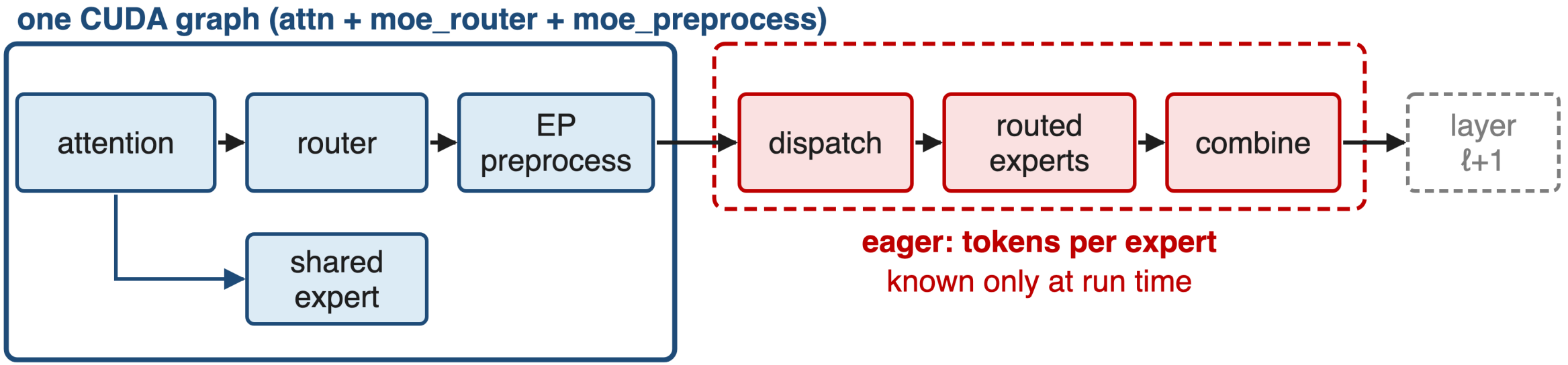
Automatic: the training schedule drives warmup, capture and replay

❸ transformer_engine: capture ahead in _order



TECudaGraphHelper turns the PP schedule into _order

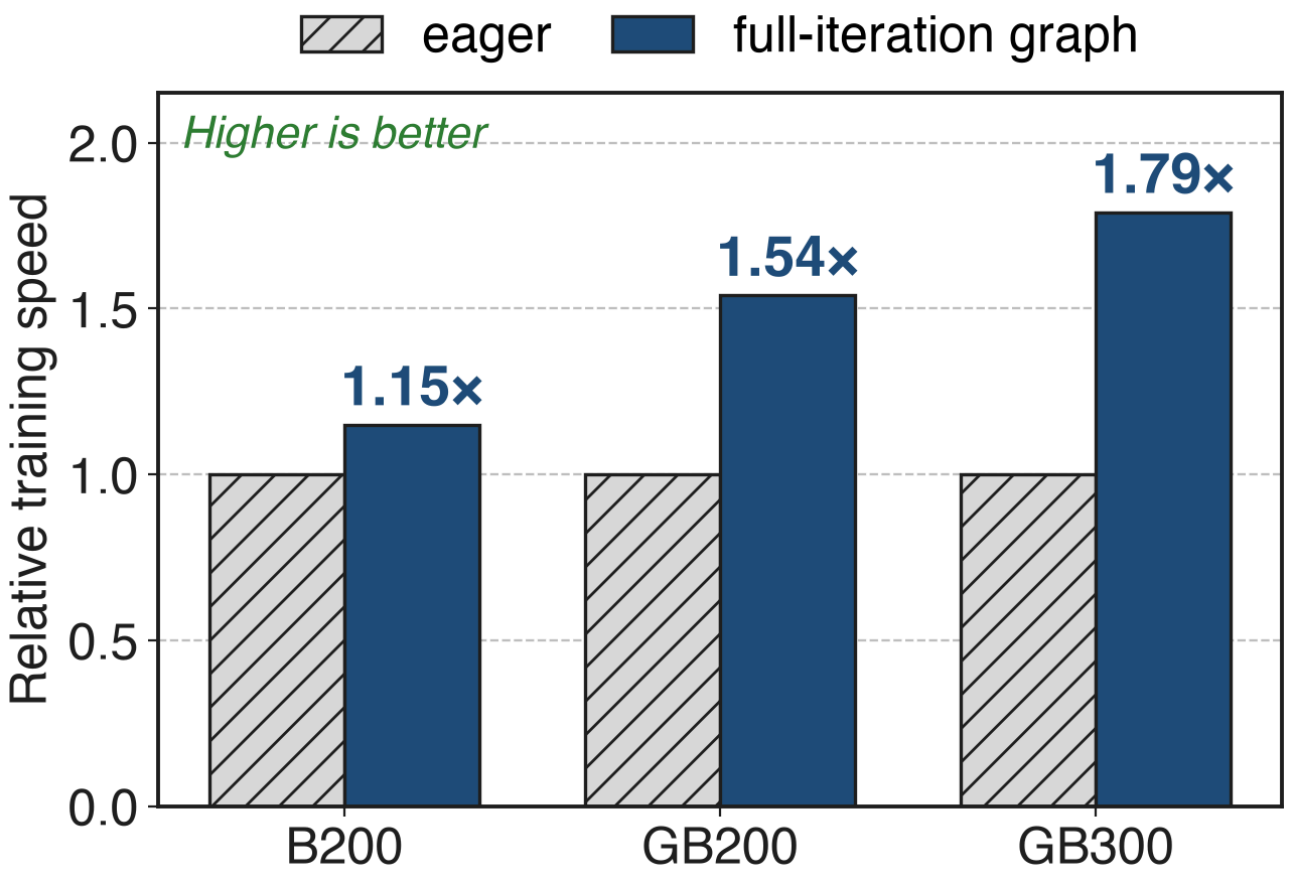
❹ MoE: graph everything before dispatch



Whole-layer dropless graphs need sync-free dispatch (deck 19)

Full-iteration graphs: up to 1.79× on FP4

- Llama 3.1 405B, 512 GPUs
- FP8 on B200/GB200: no gain
- vs per-layer: up to **19%** less memory



MLPerf Training v5.1, NVIDIA dl-cuda-graph docs (Llama 3.1 405B example)

Partial graphs add 10% to DeepSeek-V3

+10%

end-to-end speedup for DeepSeek-V3 training on GB200 with partial CUDA graphs, at about 7 GB extra memory

arXiv 2603.07685 §4.3.6 (p.42); part of the 1,048 TFLOPS/GPU GB200 configuration (Table 17)

Graphs cost memory, warmup and flexibility

- **Memory**: static buffers, separate pools
- Eager warmup (3 steps), then capture
- Opaque: harder to debug and profile
- No host syncs: NaN check off, TE RNG
- Dropless experts stay eager by default
- Long GEMMs gain little (H100 DSV3: off)

Summary

- Eager FP4 Llama 405B on GB300: **1.79×** slower
- **Insight**: graph the largest static scope, replay in capture order
- full_iteration, per-layer (local or TE), MoE partial scopes
- **1.15–1.79×** full-iteration; **+10%** DeepSeek-V3 for ~7 GB

https://github.com/NVIDIA/Megatron-LM